Visualization of Sound Spectra of a Whistle

Jorge R. Gherson

April 10, 2026

Contents

1 General Objective **3**

2 Exploratory Analysis of the Sound Spectra Dataset **3**

3 Mathematical Formulation **5**

4 Software Implementation Pipeline **7**

4.1 Configuration Management (`config.py`) . . . 7
4.1.1 Key Components: . . . 7
4.2 Data Preprocessing (`data_preprocessing.py`) . . . 7
4.2.1 Processing Steps: . . . 7
4.3 Dataset Creation (`dataset.py`) . . . 9
4.3.1 Key Components: . . . 9
4.4 Model Architecture (`models.py`) . . . 10
4.4.1 WhistleCNN Architecture: . . . 10
4.5 Extended Model Architectures (`models_2d.py`) . . . 11
4.5.1 WhistleCNN1D Architecture (1D PSD): . . . 11
4.5.2 WhistleCNN2D Architecture (2D STFT): . . . 11
4.5.3 Architectural Comparison: . . . 12
4.6 Model Training (`train.py`) . . . 12
4.6.1 Training Configuration: . . . 12
4.6.2 Training Loop Structure: . . . 12
4.7 Parallel Training Pipeline (`train_parallel.py`) . . . 13
4.7.1 Training Workflow: . . . 13
4.7.2 Performance Metrics: . . . 13
4.7.3 Output Artifacts: . . . 13
4.7.4 Baseline Models (`baselines.py`) . . . 13
4.7.5 Baseline 1: Peak Frequency Linear Regression . . . 13
4.7.6 Baseline 2: Full Spectrum Ridge Regression . . . 13
4.7.7 Evaluation Metrics: . . . 14
4.7.8 Visualization: . . . 14
4.8 Tree-Based Models (`tree_models.py`) . . . 14
4.8.1 Model 1: Random Forest Regressor . . . 14
4.8.2 Model 2: XGBoost Regressor . . . 14
4.8.3 Feature Importance Visualization: . . . 14
4.9 Visualization (`visualizations.py`) . . . 14
4.9.1 set_plot_style(): . . . 15
4.9.2 plot_comparative_spectra(): . . . 15
4.9.3 plot_global_spectrogram(): . . . 15
4.9.4 Output: . . . 15
4.10 Comparative Visualization Framework (`comparison_viz.py`) . . . 15
4.10.1 PercentileSampler Class: . . . 15
4.10.2 plot_percentile_grid(): . . . 16
4.10.3 plot_contrast_pairs(): . . . 16
4.10.4 plot_all_percentiles_overlay(): . . . 16
4.10.5 plot_stft_heatmap_progression(): . . . 16
4.10.6 Output Visualizations: . . . 16
4.11 Execution Order and Data Flow . . . 17
4.11.1 Data Flow Diagram: . . . 18
4.12 Key Design Decisions . . . 18

1 General Objective

We have a dataset containing the sound spectra of a whistle collected at different mass flow rates, pressures, and frequencies. The main objective of this project is to visualize the sound spectra in a way that would allow a Convolution Neural Network (CNN) or a human to efficiently see the relationship between the sound spectra and the mass flow rate.

2 Exploratory Analysis of the Sound Spectra Dataset

The initial data analysis was carried out in a Jupyter notebook named `initial_analysis.ipynb`. First, it is necessary to import the required libraries and load the dataset. The dataset was given in a `.mat` file, which was then loaded using the `scipy.io` library. We have a dataset that contains 267 measurement instances for the following variables:

1. P_{all}: A 2D array containing the sound pressure (in Pascals) for each frequency and time point.
2. `amplitude_all`: A 2D array containing the amplitude of the sound for each frequency and time point.
3. `frequency_all`: A 2D array containing the Fast Fourier Transform (FFT) frequencies with 49902 points.
4. `m_dot_all`: A 2D array containing the mass flow rate (in kg/s) for each frequency and time point.
5. `signal_all`: A 2D array containing the 1 second worth of the original microphone signal collected at 100,0000 Hz.

First, I sought to understand the structure of the dataset by checking the shapes of each variable. The shapes of the variables are as follows:

```
P_all (267, 1)
amplitude_all (267, 1)
frequency (49902, 1)
m_dot_all (267, 1)
signal_all (267, 1)
```

Notice all of the lengths agree, except for the frequency variable, which has a different shape. This is because the frequency variable contains the FFT frequencies, which are the same for all samples, while other variables contain data for each sample. Then, I evaluated if the 267 amplitudes correspond with every Nth frequency, where N is the number of frequencies. Finding the ratio between data points in the frequency variable and the amplitude variable gave us a ratio of 186. This means that the amplitude variable is not evenly distributed across the frequencies, which is an important observation for the visualization process.

Then, I proceeded to unpack each variable and asign it to a respective one in Python to evaluate how the data was structured, finding that data stored in the `.mat` file is passed as a dictionary in Python (Note: not sure.). Running for `amplitude_all` yielded:

```
array([[array([[8.72111064e-05],
            [3.07487090e-04],
            [2.17788468e-04],
            ...,
            [4.34430331e-06],
            [4.19685731e-06],
            [1.59920000e-06]], shape=(49902, 1))],
    [array([[3.06550360e-04],
            [6.16053321e-04],
            [2.33531321e-04],
            ...,
            [3.32547277e-06],
            [1.64016323e-06],
            [9.15600000e-07]], shape=(49902, 1))],
```

```
        [array([[1.70402252e-04],
                [1.92546836e-04],
                [2.33929164e-04],
                ...,
                [2.84986090e-06],
                [2.47268173e-06],
                [1.11950000e-06]], shape=(49902, 1))],
        [array([[3.62037875e-04],
                [3.37830354e-04],
                [1.09858487e-03],
                ...,
...
                [9.88551412e-04],
                ...,
                [3.28158474e-05],
                [3.24278557e-05],
                [3.83440000e-06]], shape=(49902, 1))]], dtype=object)
```

Running so for `signal_all` yielded similar results. Notice that both objects have a shape (49902, 1), so it makes sense to make the spectrum 1D and adding them vertically. For the amplitudes:

```
amplitude_all = np.vstack([
        amplitude_all[i].flatten()
        for i in range(len(amplitude_all))
])
```

And for the frequecies:

```
signal_all = np.vstack([
        sig.flatten()
        for sig in signal_all
])
```

Which yielded:

```
Amplitude Shape: (267, 49902)
Amplitude Dimension: 2
Amplitude dtype: float64
```

and

```
Signal Shape: (267, 100000)
Signal Dimension: 2
Signal Data Type: float64
```

Now it is necessary to note that, for frequency analysis it is convenient to implement a logarithmic scale, whhich was calculated using:

$$\log(\nu) = 20 \log\left(A + 1 \times 10^{-12}\right) \tag{1}$$

where ν represents the frequency and A the amplitude. We also add 1×10^{-12} to take noise into account. These were then normalized using

$$\nu_{\text{norm}} = \log \nu - \max \log \nu. \tag{2}$$

Then, peak frequencies were extracted by implementing masking to crop the data. In order to visualize the relationsip, we must sort the mass flow rates and frequencies. Then, I defined a sampling rate of 48000 Hz, which I used as a parameter for the Fast Fourier Transform (FFT) applied using Welch's method. Then, I converted the power to decibels (dB) using a logarithmic scale, and I normalized them relative to the maximum. These were afterwards sorted, and plotted using a heatmap that yielded:

Notice for this graph that, if the oscillations

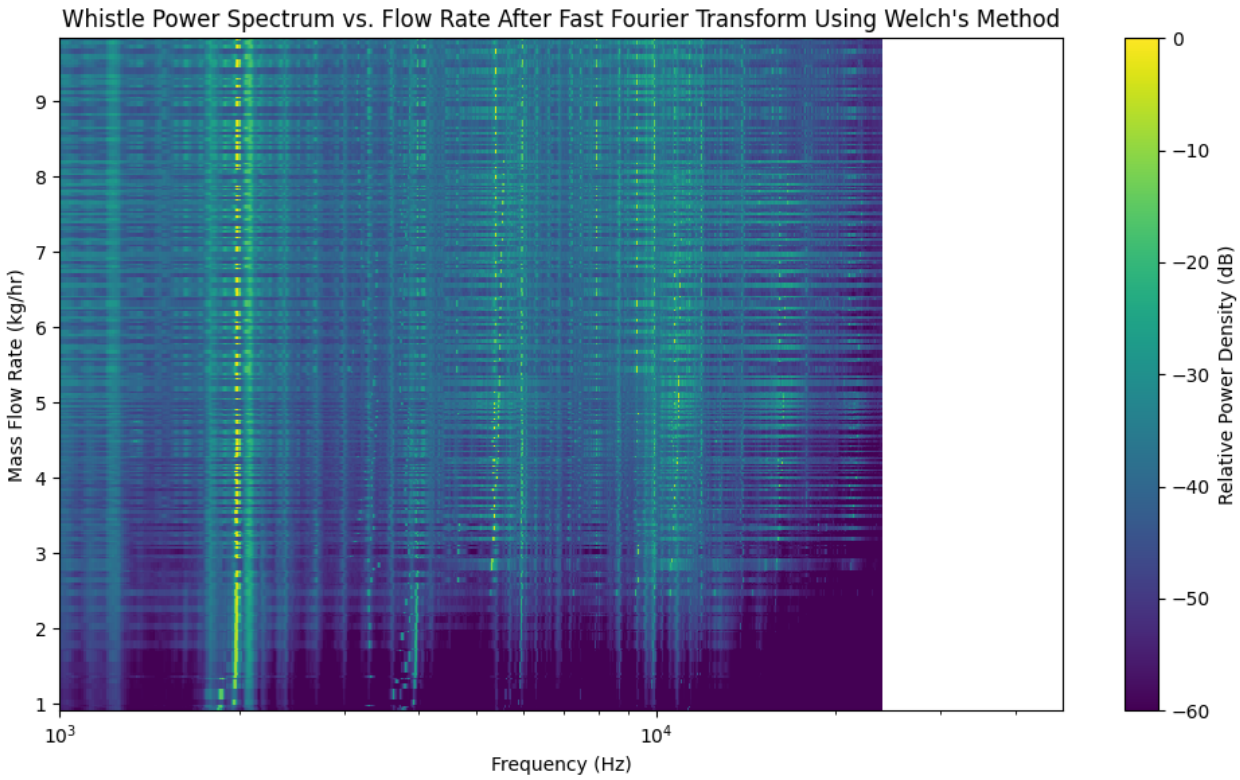

Figure 1: The spectrum plotted against the mass flow rate in a logarithmic scale after having applying Welch's method for FFTs.

3 Mathematical Formulation

In order to make the relationship easily visible for a Convolution Neural Network, I will carry out a mathematical definition for how the network transforms the raw acoustic spectrum into a mass flow rate prediction.

Let our normalized acoustic spectrum for a single flow rate be defined as a one-dimensional vector

$$\mathbf{x} = \begin{bmatrix} x_1 & x_2 & \cdots & x_N \end{bmatrix}^T \tag{3}$$

where N is the total number of frequency bins. The core operation of a CNN is the discrete 1D convolution, which applies a set of K trainable kernels (i.e., filters) to the input to detect features like whistle peaks or harmonic intervals.

Let a single filter be a vector $\mathbf{w}^{(k)}$ of length M. The convolution operation computes a new sequence $\mathbf{z}^{(k)}$ for this filter. The i-th position is given by:

$$z_i^{(k)} = \sum_{j=1}^{M} \mathbf{x}_{i+j-1} \cdot \mathbf{w}_j^{(k)} + b^{(k)} \tag{4}$$

where $\mathbf{w}_j^{(k)}$ represents the weights of the k-th filter, and $b^{(k)}$ the bias of the k-th filter. This represents a mathematical cross-correlation. When the shape of the filter matches the shape of a peak in the spectrum, z_i spikes. To allow the network to learn complex, non-linear physical relationships, we apply a Rectified Linear Unit (ReLU) to the output of the convolution:

$$h_i^{(k)} = \max\left(0, z_i^{(k)}\right). \tag{5}$$

Then, pooling reduces the size of the data, making the model computationally efficient and "translation invariant." The maximum pooling takes the maximum value over a sliding window of size P:

$$p_i^{(k)} = \max h_{P(j-1)+1}^{(k)}, \cdots, h_{Pj}^{(k)}. \tag{6}$$

Later in the network, we use global average pooling (GAP), which takes the average of the entire remaining sequence of length L for each filter k, resulting in a single scalar value per filter:

$$g^{(k)} = \frac{1}{L} \sum_{i=1}^{L} h_i^{(k)}, \tag{7}$$

which flattens the matrix into a vector $\mathbf{g} = \begin{bmatrix} g^{(1)} & g^{(2)} & \cdots & g^{(K)} \end{bmatrix}^T$. Once the spatial features (i.e., peaks, harmonics) are extracted and flattened into the vector $\mathbf{g}$, a standard neural network layer combines them to predict the flow rate. Let $\mathbf{W}_d$ be the weight matrix of the dense layer and b_d be the bias vector of the dense layer. Then,

$$\mathbf{d} = \max\left(0, W_{d\mathbf{g}} + b_d\right). \tag{8}$$

Since we are predicting a continuous physical variable, the final layer is a single node with a linear activation function:

$$\hat{y} = \mathbf{w}_{\text{out}}^T d + b_{\text{out}} \tag{9}$$

where $\hat{y}$ represents the predicted mass flow rate y and $\mathbf{w}_{\text{out}}$ represents the final weight vector mapping the learned features to the prediction. However, in order to train a network efficiently, we must also define how "wrong" our predictions can be compared to the actual mass flow rate. For regressions, we usually use Mean Squared Error (MSE) over a batch of B training samples:

$$L = \frac{1}{B} \sum_{i=1}^{B} (\hat{y}_i - y_i)^2 \tag{10}$$

4 Software Implementation Pipeline

This section provides a detailed walkthrough of how each Python file in the project implements the data processing, model training, and evaluation workflow. The software pipeline is designed to be modular, allowing each component to be executed independently or as part of a complete machine learning workflow.

4.1 Configuration Management (`config.py`)

The `config.py` file serves as the central configuration hub for the entire project. It defines all hyper-parameters, file paths, and signal processing constants in a single location, enabling easy modification without editing multiple files.

4.1.1 Key Components:

- **File Paths:** Defines `DATA_PATH` (path to the raw `.mat` file) and `PROCESSED_DIR` (directory for storing processed NumPy arrays).
- **Audio Parameters:**
  - `fs = 48000`: Sampling frequency in Hz
  - `f_min = 100`: Minimum frequency for extraction (Hz)
  - `f_max = 20000`: Maximum frequency for extraction (Hz)
  - `epsilon = 1e-12`: Small constant to prevent log(0) errors
- **Spectrogram Parameters:** Settings for scipy's spectrogram function (`spec_nperseg`, `spec_noverlap`, `spec_window`)
- **Welch's Method Parameters:** Settings for 1D PSD estimation.
  - `welch_nperseg = 4096`
  - `welch_window = 'hann'`
  - `welch_vmin_db = -120`
- **STFT Parameters:** Settings for 2D time-frequency representation.
  - `stft_nperseg = 512`: Window size for STFT (provides time-frequency tradeoff)
  - `stft_noverlap = 256`: 50% window overlap
  - `stft_window = 'hann'`
  - `stft_vmin_db = -120`: Dynamic range for noise floor clipping

The 120 dB dynamic range limit clips the noise floor, removing weak frequency components that are below the noise level and preventing the model from learning noise artifacts. This applies to both 1D PSD (Welch's method) and 2D STFT representations.

4.2 Data Preprocessing (`data_preprocessing.py`)

This file implements the core signal processing pipeline that transforms raw acoustic signals into both 1D Power Spectral Density (PSD) and 2D time-frequency representations suitable for machine learning.

4.2.1 Processing Steps:

Step 1: Load Raw Data

```
load_raw_data(filepath) -> (P_all, frequency, signal_all, amplitude_all, m_dot_all)
```

Uses `scipy.io.loadmat()` to load the MATLAB data file. The raw signals are unpacked and flattened from nested structures into 2D NumPy arrays:

- `signal_all`: Raw microphone signals at 100 kHz sampling rate – Shape (267, 100000).

- `m_dot_all`: Mass flow rates in kg/hr – Shape (267,).

General Processing Framework

Both 1D PSD and 2D STFT representations follow a common post-processing pipeline: *magnitude estimation* (if complex-valued), *logarithmic scaling*, *normalization*, and *clipping*. We define generic operations and then instantiate them for each representation.

$$X_{dB} = c \log_{10}(X + \epsilon), \tag{11}$$

where c is a scaling constant (10 for power, 20 for magnitude), X is the input, and $\epsilon = 1 \times 10^{-12}$ prevents logarithm of zero. The normalized output is given by:

$$X_{\text{norm}} = X_{dB} - \max(X_{dB}), \tag{12}$$

and the clipped output is given by:

$$X_{\text{clipped}} = \text{clip}(X_{\text{norm}}, X_{\text{min}}, 0), \tag{13}$$

where X_{min} is the minimum decibel threshold (noise floor).

Step 2: Compute Power Spectral Density using Welch's Method

```
generate_welch_psd(signal_matrix, fs) -> (frequencies, psd_normalized)
```

Applies Welch's method to reduce noise and improve PSD estimation. For each signal:

$$f, P_{xx} = \text{scipy.signal.welch}(s, fs = 48000, nperseg = 4096, window = \text{'hann'}). \tag{14}$$

The output is converted to decibels using (11) with $c = 10$ (power):

$$P_{xx,dB} = 10 \log_{10}(P_{xx} + \epsilon). \tag{15}$$

Then normalized using (12):

$$P_{xx,\text{norm}} = P_{xx,dB} - \max(P_{xx,dB}). \tag{16}$$

Finally, clipped using (13) with $X_{\text{min}} = -120$ dB:

$$P_{xx,\text{clipped}} = \text{clip}(P_{xx,\text{norm}}, -120, 0). \tag{17}$$

Step 3: Frequency Range Filtering

The processed PSD features are filtered to only include frequencies between `f_min` and `f_max` using boolean masking:

```
mask = (freqs >= config.f_min) & (freqs <= config.f_max)
psd_features_cropped = psd_features[:, mask]
```

This reduces the feature dimensionality from ≈ 12000 bins to 1698 bins (for the default configuration).

Step 4: Generate STFT Spectrograms for 2D Representation

```
generate_stft_spectrograms(signal_matrix, fs) -> (frequencies, times, stft_normalized)
```

Generates Short-Time Fourier Transform spectrograms for 2D time-frequency analysis. For each signal:

$$f, t, Z_{xx} = \text{scipy.signal.stft}(s, fs = 48000, nperseg = 512, noverlap = 256, window = \text{'hann'}). \tag{18}$$

The complex-valued STFT is converted to magnitude spectrum:

$$|Z_{xx}| = \sqrt{\text{Re}(Z_{xx})^2 + \text{Im}(Z_{xx})^2}. \tag{19}$$

Then converted to decibels using Eq. (11) with $c = 20$ (magnitude):

$$Z_{xx,dB} = 20 \log_{10}(|Z_{xx}| + \epsilon). \tag{20}$$

Normalized using Eq. (12):

$$Z_{xx,\text{norm}} = Z_{xx,dB} - \max(Z_{xx,dB}). \tag{21}$$

Finally, clipped using Eq. (13) with $X_{\min} = -120$ dB:

$$Z_{xx,\text{clipped}} = \text{clip}(Z_{xx,\text{norm}}, -120, 0). \tag{22}$$

Output Format: The STFT produces a 3D array with shape `Samples, FrequencyBins, TimeFrames`:

- Samples: 267
- Frequency Bins: 212 (after frequency range filtering)
- Time Frames: 392 (based on 512-point windows with 256-point overlap)

Complete Output: Six NumPy files are saved to `processed_data/`:

- **1D PSD Data:**
  - `psd_features.npy`: PSD matrix for all samples – Shape (267, 1698)
  - `frequencies.npy`: Corresponding frequency bins – Shape (1698,)
- **2D STFT Data:**
  - `stft_features.npy`: Time-frequency spectrograms – Shape (267, 212, 392)
  - `stft_frequencies.npy`: Frequency axis – Shape (212,)
  - `stft_times.npy`: Time axis – Shape (392,)
- `mass_flow_labels.npy`: Target labels – Shape (267,)

Step 5: Frequency Range Filtering
Both 1D and 2D features are filtered to only include frequencies between `f_min` and `f_max`.

4.3 Dataset Creation (`dataset.py`)

This file bridges NumPy arrays and PyTorch's deep learning framework, creating data loaders for efficient batch processing during model training. It supports both 1D PSD and 2D STFT representations.

4.3.1 Key Components:

WhistleDataset Class (1D PSD): A custom PyTorch `Dataset` subclass that:

1. Loads preprocessed 1D PSD NumPy arrays
2. Converts them to PyTorch tensors with dtype `float32`
3. Adds a channel dimension via `unsqueeze(1)`
   Shape transformation: $(267, 1698) \rightarrow (267, 1, 1698)$
   This is required because 1D CNNs expect $(Batch, Channels, Length)$ format.

WhistleDataset2D Class (2D STFT): A custom PyTorch `Dataset` subclass for STFT spectrograms:

1. Loads preprocessed 2D STFT NumPy arrays
2. Converts them to PyTorch tensors with dtype `float32`

3. Adds a channel dimension via `unsqueeze(1)`

   Shape transformation: $(267, 212, 392) \rightarrow (267, 1, 212, 392)$

   This is required because 2D CNNs expect $(Batch, Channels, Height, Width)$ format, where Height = Frequency bins and Width = Time frames.

get_dataloaders() Function (1D PSD):

```
train_loader, test_loader = get_dataloaders(batch_size=16, test_size=0.2)
```

Performs 80/20 train-test split and wraps datasets in `DataLoader` objects with `shuffle=True` to prevent the model from learning the order of examples.

get_dataloaders_2d() Function (2D STFT):

```
train_loader_2d, test_loader_2d = get_dataloaders_2d(batch_size=16, test_size=0.2)
```

Performs 80/20 train-test split for 2D STFT data with the same configuration as the 1D dataloader.

Output Format (1D): Each batch contains:

- `features`: Tensor of shape $(16, 1, 1698)$
- `labels`: Tensor of shape $(16, 1)$

Output Format (2D): Each batch contains:

- `features`: Tensor of shape $(16, 1, 212, 392)$ (Batch, Channels, Freq, Time)
- `labels`: Tensor of shape $(16, 1)$

4.4 Model Architecture (`models.py`)

This file defines the neural network architecture that maps acoustic spectra to mass flow rate predictions.

4.4.1 WhistleCNN Architecture:

The model consists of two main components:

Convolutional Feature Extractor: A sequence of three convolutional blocks, each containing:

1. **1D Convolution**: Applies learnable filters to detect spectral features

$$\text{output}_i = \sum_{j=0}^{K-1} w_j \cdot \text{input}_{i+j} + b \tag{23}$$

   where $K = 5$ (kernel size)

2. **ReLU Activation**: Introduces non-linearity

$$h = \max(0, x) \tag{24}$$

3. **Max Pooling**: Downsamples feature maps with stride of 2

$$\text{pool}_i = \max(\text{conv}_{2i}, \text{conv}_{2i+1}) \tag{25}$$

Block	In Channels	Out Channels	Output Size
Input	—	—	$(B, 1, 1698)$
Conv Block 1	1	16	$(B, 16, 849)$
Conv Block 2	16	32	$(B, 32, 424)$
Conv Block 3	32	64	$(B, 64, 212)$
Adaptive Avg Pool	64	64	$(B, 64, 32)$

The `AdaptiveAvgPool1d(32)` layer ensures that regardless of input size, the output is always 64 channels $\times$ 32 frequency bins = 2048 features.

Fully Connected Regressor:

```
Linear(2048, 128) -> ReLU -> Dropout(0.3) -> Linear(128, 32) -> ReLU -> Linear(32, 1)
```

- Layer 1: Maps 2048 features to 128 hidden units
- Dropout: Randomly zeros 30% of activations during training to prevent overfitting
- Layer 2: Maps 128 hidden units to 32 hidden units
- Output Layer: Single neuron with linear activation for continuous regression

Model Complexity: Total parameters: $\approx 367,000$

4.5 Extended Model Architectures (`models_2d.py`)

This file defines both 1D and 2D CNN architectures for parallel model comparison. It enables systematic evaluation of whether temporal/spectrographic information improves prediction accuracy over simple PSD features.

4.5.1 WhistleCNN1D Architecture (1D PSD):

Identical to the model in `models.py`, this baseline uses only frequency information:

- Input: $(Batch, 1, 1698)$ (PSD features)
- Total Parameters: $\approx 279,649$

4.5.2 WhistleCNN2D Architecture (2D STFT):

A novel 2D convolutional architecture designed for time-frequency spectrograms:

Convolutional Feature Extractor: A sequence of four 2D convolutional blocks, each containing:

1. **2D Convolution**: Applies learnable filters to detect spectrotemporal patterns

$$\text{output}_{i,j} = \sum_{u,v} w_{u,v} \cdot \text{input}_{i+u,j+v} + b \tag{26}$$

   where kernel size $K = 5 \times 5$

2. **Batch Normalization**: Normalizes activations to accelerate training

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} \tag{27}$$

3. **ReLU Activation**: Introduces non-linearity (see (24))
4. **Max Pooling**: Downsamples feature maps with stride of 2 (see Eq. (25))

Block	In Ch.	Out Ch.	Kernel	Output Size
Input	—	—	—	$(B, 1, 212, 392)$
Conv Block 1	1	16	5×5	$(B, 16, 106, 196)$
Conv Block 2	16	32	5×5	$(B, 32, 53, 98)$
Conv Block 3	32	64	3×3	$(B, 64, 26, 49)$
Conv Block 4	64	128	3×3	$(B, 128, 13, 24)$
Adaptive Avg Pool	128	128	—	$(B, 128, 16, 16)$

The `AdaptiveAvgPool2d((16, 16))` layer ensures that regardless of input dimensions, the output is always $128 \times 16 \times 16 = 32,768$ features.

Fully Connected Regressor:

```
Linear(32768, 256) -> ReLU -> Dropout(0.4) ->
Linear(256, 128) -> ReLU -> Dropout(0.3) ->
Linear(128, 64) -> ReLU -> Dropout(0.2) ->
Linear(64, 1)
```

Multi-layer design with progressive dropout rates to prevent overfitting on high-dimensional input.

Model Complexity: Total parameters: $\approx 8,536,161$ (30.5× more than 1D model)

4.5.3 Architectural Comparison:

Aspect	1D CNN	2D CNN
Input Shape	$(B, 1, 1698)$	$(B, 1, 212, 392)$
Input Type	1D PSD (frequency only)	2D STFT (time-frequency)
Total Parameters	279,649	8,536,161
Conv Dimension	1D filters	2D filters
Feature Extraction	Spectral patterns	Spectrotemporal patterns

4.6 Model Training (`train.py`)

This file implements the training loop, loss computation, and model checkpointing.

4.6.1 Training Configuration:

- **Device Setup**: Uses GPU (CUDA/MPS) if available, falls back to CPU
- **Loss Function**: L1Loss (Mean Absolute Error)

$$L = \frac{1}{B} \sum_{i=1}^{B} |\hat{y}_i - y_i| \tag{28}$$

  L1 loss is robust to outliers and matches the evaluation metric (MAE).

- **Optimizer**: Adam with learning rate $\alpha = 0.001$
- **Epochs**: 50
- **Batch Size**: 16

4.6.2 Training Loop Structure:

For each epoch:

1. **Training Phase:**

```
for features, labels in train_loader:
    predictions = model(features)
    loss = criterion(predictions, labels)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

2. **Validation Phase:**

```
with torch.no_grad():
    for features, labels in test_loader:
        predictions = model(features)
        val_loss = criterion(predictions, labels)
```

3. **Model Checkpointing:** If validation MAE improves, save model weights:

```
if epoch_val_loss < best_val_loss:
    torch.save(model.state_dict(),
               "saved_models/best_whistle_cnn.pth")
```

Output:

- Trained model saved to `saved_models/best_whistle_cnn.pth`
- Learning curves plotted (training MAE vs. validation MAE over epochs)

4.7 Parallel Training Pipeline (`train_parallel.py`)

This file implements side-by-side training and comparison of 1D and 2D models to evaluate which representation is most suitable for the task.

4.7.1 Training Workflow:

1. **Train 1D CNN** using PSD features from `dataset.get_dataloaders()`
2. **Train 2D CNN** using STFT features from `dataset.get_dataloaders_2d()`
3. **Compare Performance** across multiple metrics
4. **Generate Reports** with visualizations and analysis

4.7.2 Performance Metrics:

Both models are evaluated on:

- **Best Validation MAE**: Lowest mean absolute error achieved
- **Convergence Epoch**: Epoch at which best performance occurred
- **Parameter Efficiency**: Parameters per unit of performance improvement
- **Final Validation MAE**: Error after all training epochs
- **Training Stability**: Variation in validation loss over epochs

4.7.3 Output Artifacts:

- `training_comparison_individual.png`: Separate training curves for each model
- `training_comparison_overlay.png`: Overlaid validation loss comparison
- `performance_report.txt`: Detailed quantitative comparison
- `saved_models/best_whistle_cnn_1d.pth`: Best 1D model checkpoint
- `saved_models/best_whistle_cnn_2d.pth`: Best 2D model checkpoint

4.7.4 Baseline Models (`baselines.py`)

This file implements two machine learning baselines for comparison with the CNN model.

4.7.5 Baseline 1: Peak Frequency Linear Regression

Feature Extraction: For each spectrum, extract the frequency bin with maximum amplitude:

$$f_{\text{peak}} = f[\arg\max_i P_{xx}[i]] \tag{29}$$

Model: Simple linear regression:

$$\hat{y} = w \cdot f_{\text{peak}} + b \tag{30}$$

Rationale: Tests whether a simple linear relationship exists between peak frequency and mass flow rate.

4.7.6 Baseline 2: Full Spectrum Ridge Regression

Feature: Uses all 1698 frequency bins as input features.

Model: Ridge Regression with $\alpha = 10.0$ (L2 regularization):

$$\hat{y} = \mathbf{w}^T\mathbf{P} + b, \quad \text{with penalty } \lambda\|\mathbf{w}\|_2^2 \tag{31}$$

Ridge regularization prevents overfitting by penalizing large weight magnitudes, especially important when features are highly correlated (which is typical for adjacent frequency bins).

4.7.7 Evaluation Metrics:

Both baselines and the CNN are evaluated using:

- **Mean Squared Error (MSE)**:

$$\text{MSE} = \frac{1}{B}\sum_i (\hat{y}_i - y_i)^2 \tag{32}$$

- **Mean Absolute Error (MAE)** (in kg/hr):

$$\text{MAE} = \frac{1}{B}\sum_i |\hat{y}_i - y_i| \tag{33}$$

- **R^2 Score**:

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2} \tag{34}$$

4.7.8 Visualization:

Parity plot compares predicted vs. actual mass flow rates for both baselines, showing prediction accuracy.

4.8 Tree-Based Models (`tree_models.py`)

This file trains two ensemble learning models using scikit-learn.

4.8.1 Model 1: Random Forest Regressor

Configuration:

```
RandomForestRegressor(n_estimators=100, random_state=42, n_jobs=-1)
```

- **n_estimators**: 100 decision trees
- **n_jobs=-1**: Use all available CPU cores for parallel training
- Each tree learns different aspects of the frequency-flow relationship

4.8.2 Model 2: XGBoost Regressor

Configuration:

```
XGBRegressor(n_estimators=100, learning_rate=0.1, max_depth=4, random_state=42)
```

- **learning_rate=0.1**: Shrinkage parameter to prevent overfitting
- **max_depth=4**: Shallow trees for reduced model complexity
- Gradient boosting sequentially builds trees to correct errors

4.8.3 Feature Importance Visualization:

Both models provide `feature_importances_` attribute:

$$\text{importance}_j = \frac{\text{number of splits using feature } j}{\text{total splits}} \tag{35}$$

The top 10 most important frequencies are visualized, revealing which spectral regions are most predictive of mass flow rate.

4.9 Visualization (`visualizations.py`)

This file creates plots from processed data.

4.9.1 set_plot_style():

Globally configures matplotlib with consistent formatting:

- Font sizes, DPI (300), grid style
- Removes top and right spines for cleaner appearance
- Uses sans-serif fonts

4.9.2 plot_comparative_spectra():

Creates a line plot comparing two spectra:

- **Low Flow Spectrum**: Minimum mass flow rate sample

$$i_{\min} = \arg\min_i m_{\text{dot}}[i] \tag{36}$$

- **High Flow Spectrum**: Maximum mass flow rate sample

$$i_{\max} = \arg\max_i m_{\text{dot}}[i] \tag{37}$$

Both spectra are plotted on semi-log axes (x-axis log scale), revealing how acoustic signatures change with flow rate. Saved to `plots/spectra_comparison.png`.

4.9.3 plot_global_spectrogram():

Creates a 2D heatmap (pcolormesh) with:

- **X-axis**: Frequencies (log scale, Hz)
- **Y-axis**: Mass flow rates (kg/hr), sorted ascending
- **Color**: Power Spectral Density (dB), using 'magma' colormap
- **Range**: -60 dB (floor) to 0 dB (normalized max)

This global spectrogram reveals how the spectral content evolves smoothly with increasing mass flow rate.

4.9.4 Output:

All plots saved to `plots/` directory at 300 DPI for publication quality.

4.10 Comparative Visualization Framework (`comparison_viz.py`)

This file creates comprehensive side-by-side comparisons of 1D PSD and 2D STFT representations with percentile-based sampling for efficient data exploration.

4.10.1 PercentileSampler Class:

Systematically samples data at quantile points for contrasting visualization:

$$p \in \{5, 25, 50, 75, 95\} \tag{38}$$

For each percentile, finds the nearest sample:

$$i_p = \arg\min_i |m_{\text{dot}}[i] - \text{percentile}_p(m_{\text{dot}})| \tag{39}$$

Also provides contrast pairs:

- Pair 1: 5th vs 95th percentile (extreme contrast)
- Pair 2: 25th vs 75th percentile (moderate contrast)

4.10.2 plot_percentile_grid():

Creates a 5×2 grid visualization:

- **Rows**: Five percentile levels (5th, 25th, 50th, 75th, 95th)
- **Column 1**: 1D PSD spectrum (line plot, semi-log axis)
- **Column 2**: 2D STFT spectrogram (pcolormesh heatmap)

Each row displays the same sample in both representations, enabling direct visual comparison.

4.10.3 plot_contrast_pairs():

Creates detailed side-by-side comparisons for extreme cases:

- Figure for each contrast pair (2 figures total)
- Top row: Low mass flow sample
- Bottom row: High mass flow sample
- Each shows 1D PSD and 2D STFT representations

Highlights acoustic signature changes between extreme flow conditions.

4.10.4 plot_all_percentiles_overlay():

Overlays all 5 percentile samples on a single plot:

- X-axis: Frequency (Hz)
- Y-axis: Power (dB)
- Five curves with color gradient (RdYlBu_r colormap)
- Two variants: logarithmic and linear frequency scales

Shows smooth progression of spectral content across flow range.

4.10.5 plot_stft_heatmap_progression():

Displays all 5 percentile STFT spectrograms in a single row:

- Five subplots, one per percentile
- Each shows frequency (vertical, log scale) vs. time (horizontal)
- Color represents magnitude in dB (magma colormap)
- Reveals temporal patterns across flow conditions

4.10.6 Output Visualizations:

- `percentile_grid_1d_vs_2d.png`: Main 5×2 comparison (1.6 MB)
- `contrast_pair_1_5_vs_95_Percentile.png`: Extreme contrast (393 KB)
- `contrast_pair_2_25_vs_75_Percentile.png`: Moderate contrast (528 KB)
- `percentile_overlay_log_scale.png`: 1D progression log (782 KB)
- `percentile_overlay_linear_scale.png`: 1D progression linear (842 KB)
- `stft_percentile_heatmaps.png`: 2D progression row (1.2 MB)

All visualizations saved at 300 DPI for publication quality.

4.11 Execution Order and Data Flow

The typical workflow to reproduce results is:

1. **Step 1**: Run `data_preprocessing.py`

   ```
   python data_preprocessing.py
   ```

   Outputs: `processed_data/{psd_features.npy, mass_flow_labels.npy, frequencies.npy}`

2. **Step 2 (Optional)**: Run `baselines.py`

   ```
   python baselines.py
   ```

   Provides baseline MAE/R^2 scores for comparison.

3. **Step 3 (Optional)**: Run `tree_models.py`

   ```
   python tree_models.py
   ```

   Trains Random Forest and XGBoost, displays feature importances.

4. **Step 4**: Run `train.py`

   ```
   python train.py
   ```

   Trains CNN and saves best model to `saved_models/best_whistle_cnn.pth`

5. **Step 5 (Optional)**: Run `comparison_viz.py`

   ```
   python comparison_viz.py
   ```

   Generates comprehensive 1D vs 2D visualization grids with percentile sampling.

6. **Step 6 (Optional)**: Run `train_parallel.py`

   ```
   python train_parallel.py
   ```

   Trains both 1D and 2D models in sequence, generates comparison plots and performance report.

7. **Step 7 (Optional)**: Run `visualizations.py`

   ```
   python visualizations.py
   ```

   Generates publication-quality plots from original visualization pipeline.

4.11.1 Data Flow Diagram:

4.12 Key Design Decisions

1. **Modular Structure**: Each file handles a specific responsibility (preprocessing, modeling, evaluation), making the codebase testable and maintainable.

2. **Configuration Centralization**: All hyperparameters in `config.py` enable rapid experimentation without editing multiple files. Both 1D (Welch) and 2D (STFT) parameters are configurable.

3. **Dual Representation Pipeline**: Generating both 1D PSD and 2D STFT representations enables systematic comparison of feature extraction strategies. The STFT provides time-frequency localization while Welch's method provides noise-robust frequency estimation.

4. **Welch's Method for 1D**: Window size of 4096 with 50% overlap (2048-point) reduces noise and computational cost compared to raw FFT while providing sufficient frequency resolution.

5. **STFT Parameters for 2D**: Smaller 512-point window with 256-point (50%) overlap balances time and frequency resolution for spectrotemporal feature extraction. Produces 212 frequency bins and 392 time frames per signal.

6. **Percentile-Based Sampling**: Systematic sampling at 5th, 25th, 50th, 75th, and 95th percentiles provides efficient coverage of the data range without exhaustively visualizing all 267 samples.

7. **Parallel Model Comparison**: Training 1D and 2D models sequentially enables direct performance comparison and guides architecture selection based on empirical results rather than assumptions.

8. **L1 Loss for Training**: More robust to outliers than L2 loss and directly optimizes the evaluation metric (MAE).

9. **Adaptive Pooling**: Ensures both 1D and 2D CNNs work with variable input sizes without requiring fixed-size inputs.

10. **Progressive Dropout**: Increasing dropout rates from layer 2 (0.4) to layer 4 (0.2) in the 2D model applies stronger regularization to early layers where overfitting is more likely with high-dimensional inputs.

11. **Multiple Representation Comparison**: Systematic visualization of 1D vs 2D representations across percentile samples enables qualitative assessment of which features are captured by each approach.

12. **Multiple Models**: Baselines and tree models provide context for CNN performance and reveal which frequencies matter most via feature importance rankings.